

Name and student number:

Yuhan Fan

1011119415

The purpose of the project:

This project builds a very basic tickets managing system using database combined with JDBC. It only allows non-parallel and non-concurrent operations and interacts with the user via the terminal. Although it's very far from a real application, a real application may be built on top of it.

Conceptual problems and my solutions:

1. My first design uses a boolean "cancelled" to record if a Performance or Ticket is cancelled. However, the report part R6 requires me to analyze the cancellations "within a year". To record the information of when the cancellation happened, I changed the boolean to a timestamp "cancel_datetime", which records 2 information: (1) Is it cancelled? (2) When, if it's cancelled?
2. When the profile of a User should be deleted, the history of his organizing or purchasing should remain in the database. Then the User cannot disappear completely. Thus, I added a "deleted" in relation User, and the Users marked as deleted is unavailable for any new operations, however past Events may still point to it as organizer, or past Orders may still point to it as the customer.
3. In my ER diagram and relation schema, "Seats" means physical seats, and "General_sections" means physical general sections. A problem is, if a Reserved_ticket is related with a Seat, or a General_ticket is related with a General_section, how does it relate to the conceptual seat/section in a specific performance as they should? The answer is, the Tickets records Orders OID, and Orders records Performance PID, so given a Ticket there is a way to know its assigned space in a performance.
4. Another problem is whether a Price tier belongs to a physical section in a Venue, or a Performance. According to the handout, the same physical section may be assigned different price tiers in different performances. So my solution let Price_tiers be a weak entity set under Performances. It has 1-many relationship with physical Sections.
5. I also thought about what "Block" (blocking a reserved seat in a performance) should contain. In my solution, "Block" doesn't record Venue name, even though Section is a weak entity set under Venue. That is because "Block" has PID, and Performance has many-1 relationship with Venues. Knowing the PID is sufficient for knowing the Venue name. My solution eliminates redundancy.
6. One hard problem is :What should a Listing look like? At first I designed Listings to have 2 relationships to Customers, 1 listing and 1 buying. The relation schema should look like, Listings(lid, price, datetime, seller_id, withdrawn), Purchase_listing(lid, buyer_id, datetime). Then I decided to combine them in 1 because 1 listing should be paired up with 1 buying, or no buying. Listings(lid, tid, seller_id, list_datetime, price, withdraw_datetime, buyer_id, trans_datetime) in my current design can store the same information, and some of (withdraw_datetime, buyer_id, trans_datetime) being NULL conveys the 3 states of a Listing – Active, Withdrawn, Sold.
7. "Most popular noun phrases" is defined as noun phrases that appear in a higher number of

Reviews, rather than appear in any Reviews at a high frequency. Because I think “popular” means most Customers who have watched the Performance have said it.

8. The algorithm for suggesting price tiers is: For any Performance P, find the most recent earlier Performance P' (already past and not cancelled) that happened in the same Venue and had the same Segment, and output the price tiers and section assignments of P' for the organizer. (If there is no such P', can't make a suggestion. But as the database gets big, it's likely that we can find a P'.) This algorithm is very simple but reasonable: simple because it works with a simple lookup rather than complex calculation, reasonable because another Performance with the same Venue and Segment can be a very good reference for this Performance.

The assumptions I did:

Assumptions on “Information to record”:

1. The names of the Venues are unique.
2. The names of the Sections are unique within the Venues.
3. Segments are unique. Genres are unique within each Segment.
4. A Listing can only be one of the states:

	sold	seller withdrawn	selling
withdrawn-datetime	NULL	√	NULL
buyer-id	√	NULL	NULL
trans-datetime	√	NULL	NULL

Assumptions on “Operations to support”:

1. When any profile is deleted, the UID becomes not accessible, but the history (e.g. organizing and purchasing history) stays in database. If the User is a Customer, all his Listings that are not withdrawn and not sold will be automatically withdrawn.
2. When an organizer cancels a performance, all the Tickets that are sold are marked cancelled, and all the Listings that are not sold nor withdrawn are automatically withdrawn.
3. Changing of Event resale-cap only affects future Listings.
4. One Customer may only buy and sell a Ticket once, because it's very unlikely in real life, that customers buy and sell tickets in a loop: A->B->C->A. (But A->B->C->D... is allowed).
5. A Ticket bought from a Listing can't be cancelled.
6. If a Customer listed a Performance's Ticket, even though nobody bought it he did not withdraw listing, then I assume he didn't go to the Performance and can't review the Performance. (If he withdraws, then I assume he went. If he has other Tickets on this Performance, he might have gone to the Performance – depends on other Tickets.)
7. Customers can only review the Performances they attended in a month, and “attend” means they hold the Ticket without cancelling or listing (only except he withdraws listing).

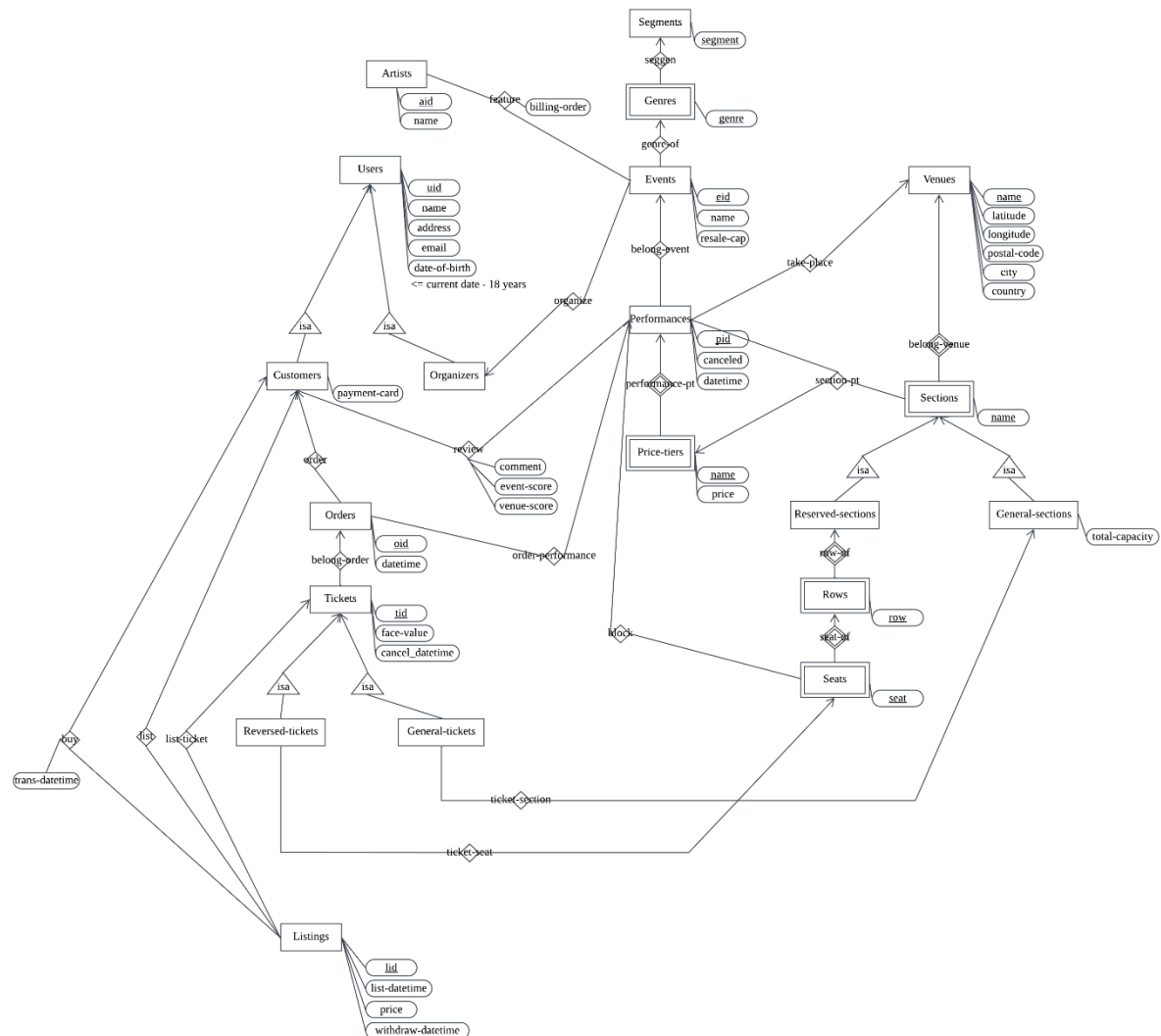
Assumptions on “Queries to support”:

1. Adjacent postal codes are postal codes that have the same first 3 characters.
2. Outputting a selected Performance to the user means providing the pid, Event name, Performance datetime, and Venue name.

Assumptions on “Reports to support”:

- For R2, if an Event has at least 1 Performance (not cancelled) happening in a particular country/city/venue, I count it as an Event happening there. Thus, an Event with Performances happening at different places may be counted multiple times.

ER diagram:



Relation schema: (Note: All the “-” are actually “_”, but “_” confuses reader with underlining, so I write “-” instead. For relations with non-trivial FDs, they’re in BCNF.)

Artists (aid, name)

FDs: aid -> name

LHS aid is primary key, thus superkey, so Artists is in BCNF.

Users (uid, name, address, email, date-of-birth, deleted)

FDs: uid -> name, address, email, date-of-birth, deleted

LHS uid is primary key, thus superkey, so Users is in BCNF.

Customers (uid, payment-card)

FDs: uid -> payment-card

LHS uid is primary key, thus superkey, so Customers is in BCNF.

Organizers (uid)

FDs:

Orders (oid, uid, pid, datetime)

FDs: oid -> uid, pid, datetime

LHS oid is primary key, thus superkey, so Orders is in BCNF.

Tickets (tid, face-value, cancel_datetime, oid)

FDs: tid -> face-value, cancel_datetime, oid

LHS tid is primary key, thus superkey, so Tickets is in BCNF.

Reserved-tickets (tid, seat, row, section-name, venue-name)

FDs: tid -> seat, row, section-name, venue-name

LHS tid is primary key, thus superkey, so Reserved-tickets is in BCNF.

General-tickets (tid, section-name, venue-name)

FDs: tid -> section-name, venue-name

LHS tid is primary key, thus superkey, so General-tickets is in BCNF.

Listings (lid, tid, seller-id, list-datetime, price, withdraw-datetime, buyer-id, trans-datetime)

FDs: lid -> tid, seller-id, list-datetime, price, withdraw-datetime, buyer-id, trans-datetime

LHS lid is primary key, thus superkey, so Listings is in BCNF.

Segments (segment)

FDs:

Genres (segment, genre)

FDs:

Events (eid, name, resale-cap, uid, genre, segment)

FDs: eid -> name, resale-cap, uid, genre, segment

LHS eid is primary key, thus superkey, so Events is in BCNF.

Performances (pid, cancel_datetime, datetime, eid, venue-name)

FDs: pid -> cancel_datetime, datetime, eid, venue-name

LHS pid is primary key, thus superkey, so Performances is in BCNF.

Price-tiers (name, price, pid)

FDs: name, pid -> price

LHS (name, pid) is primary key, thus superkey, so Price-tiers is in BCNF.

Venues (name, latitude, longitude, postal-code, city, country)

FDs: name -> latitude, longitude, postal-code, city, country

LHS name is primary key, thus superkey, so Venues is in BCNF.

Sections (name, venue-name)

FDs:

Reserved-sections(name, venue-name)

FDs:

General-sections (name, venue-name, total-capacity)

FDs: name, venue-name -> total-capacity

LHS (name, venue-name) is primary key, thus superkey, so General-sections is in BCNF.

Rows (row, section-name, venue-name)

FDs:

Seats (seat, row, section-name, venue-name)

FDs:

Feature (aid, eid, billing-order)

FDs: aid, eid -> billing-order

LHS (aid, eid) is primary key, thus superkey, so Feature is in BCNF.

Review (uid, pid, comment, event-score, venue-score)

FDs: uid, pid -> comment, event-score, venue-score

LHS (uid, pid) is primary key, thus superkey, so Review is in BCNF.

Section-pricetier (section-name, pid, pricetier-name)

FDs: section-name, pid -> pricetier-name

LHS (section-name, pid) is primary key, thus superkey

so Section-pricetier is in BCNF.

Block (pid, seat, row, section-name)

FDs:

DDL statements:

```
CREATE TABLE Artists (  
    aid            INT            AUTO_INCREMENT PRIMARY KEY,  
    name           VARCHAR(255)   NOT NULL  
);  
  
CREATE TABLE Users (  
    uid            INT            AUTO_INCREMENT PRIMARY KEY,  
    name           VARCHAR(255)   NOT NULL,  
    address        VARCHAR(255)   NOT NULL,  
    email          VARCHAR(255)   NOT NULL,  
    date_of_birth  DATE           NOT NULL,  
    deleted        BOOLEAN        NOT NULL DEFAULT false  
);  
  
CREATE TABLE Segments (  
    segment        VARCHAR(255)   PRIMARY KEY  
);  
  
CREATE TABLE Customers (  
    uid            INT            PRIMARY KEY,  
    payment_card   VARCHAR(16)   NOT NULL,  
    FOREIGN KEY (uid) REFERENCES Users(uid)  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE Organizers (  
    uid            INT            PRIMARY KEY,  
    FOREIGN KEY (uid) REFERENCES Users(uid)  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE Genres (  
    genre          VARCHAR(255)   NOT NULL,  
    segment        VARCHAR(255)   NOT NULL,  
    PRIMARY KEY (genre, segment),  
    FOREIGN KEY (segment) REFERENCES Segments(segment)  
        ON UPDATE CASCADE  
);  
  
CREATE TABLE Venues (  
    name           VARCHAR(255)   PRIMARY KEY,  
    latitude        DECIMAL(9, 6) NOT NULL,  
    longitude       DECIMAL(9, 6) NOT NULL,  
    postal_code     VARCHAR(6)    NOT NULL,  
    city            VARCHAR(255)   NOT NULL,  
    country         VARCHAR(255)   NOT NULL  
);
```

```

CREATE TABLE Events (
    eid            INT            AUTO_INCREMENT PRIMARY KEY,
    name           VARCHAR(255)   NOT NULL,
    resale_cap     DECIMAL(5, 2)  NOT NULL,
    uid            INT            NOT NULL,
    genre          VARCHAR(255)   NOT NULL,
    segment       VARCHAR(255)   NOT NULL,
    FOREIGN KEY (uid) REFERENCES Organizers(uid)
        ON UPDATE CASCADE,
    FOREIGN KEY (genre, segment) REFERENCES Genres(genre, segment)
        ON UPDATE CASCADE
);

CREATE TABLE Performances (
    pid            INT            AUTO_INCREMENT PRIMARY KEY,
    eid            INT            NOT NULL,
    cancel_datetime DATETIME      DEFAULT NULL,
    datetime       DATETIME      NOT NULL,
    venue_name     VARCHAR(255)   NOT NULL,
    FOREIGN KEY (eid) REFERENCES Events(eid)
        ON UPDATE CASCADE,
    FOREIGN KEY (venue_name) REFERENCES Venues(name)
        ON UPDATE CASCADE
);

CREATE TABLE Price_tiers (
    name           VARCHAR(255)   NOT NULL,
    pid            INT            NOT NULL,
    price          DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (name, pid),
    FOREIGN KEY (pid) REFERENCES Performances(pid)
        ON UPDATE CASCADE
);

CREATE TABLE Sections (
    name           VARCHAR(255)   NOT NULL,
    venue_name     VARCHAR(255)   NOT NULL,
    PRIMARY KEY (name, venue_name),
    FOREIGN KEY (venue_name) REFERENCES Venues(name)
        ON UPDATE CASCADE
);

CREATE TABLE Reserved_sections (
    name           VARCHAR(255)   NOT NULL,
    venue_name     VARCHAR(255)   NOT NULL,
    PRIMARY KEY (name, venue_name),
    FOREIGN KEY (name, venue_name) REFERENCES Sections(name, venue_name)

```

```

        ON UPDATE CASCADE
    );

CREATE TABLE General_sections (
    name          VARCHAR(255)      NOT NULL,
    venue_name     VARCHAR(255)      NOT NULL,
    total_capacity INT               NOT NULL,
    PRIMARY KEY (name, venue_name),
    FOREIGN KEY (name, venue_name) REFERENCES Sections(name, venue_name)
        ON UPDATE CASCADE
);

CREATE TABLE Rows (
    row           INT               NOT NULL,
    section_name  VARCHAR(255)      NOT NULL,
    venue_name     VARCHAR(255)      NOT NULL,
    PRIMARY KEY (row, section_name, venue_name),
    FOREIGN KEY (section_name, venue_name) REFERENCES Reserved_sections(name,
venue_name)
        ON UPDATE CASCADE
);

CREATE TABLE Seats (
    seat          INT               NOT NULL,
    row           INT               NOT NULL,
    section_name  VARCHAR(255)      NOT NULL,
    venue_name     VARCHAR(255)      NOT NULL,
    PRIMARY KEY (seat, row, section_name, venue_name),
    FOREIGN KEY (row, section_name, venue_name) REFERENCES Rows(row,
section_name, venue_name)
        ON UPDATE CASCADE
);

CREATE TABLE Orders (
    oid           INT               AUTO_INCREMENT PRIMARY KEY,
    uid           INT               NOT NULL,
    pid           INT               NOT NULL,
    datetime      DATETIME          NOT NULL,
    FOREIGN KEY (uid) REFERENCES Customers(uid)
        ON UPDATE CASCADE,
    FOREIGN KEY (pid) REFERENCES Performances(pid)
        ON UPDATE CASCADE
);

CREATE TABLE Tickets (
    tid           INT               AUTO_INCREMENT PRIMARY KEY,
    face_value    DECIMAL(10, 2)    NOT NULL,
    cancel_datetime DATETIME          DEFAULT NULL,

```



```

        oid          INT          NOT NULL,
        FOREIGN KEY (oid) REFERENCES Orders(oid)
        ON UPDATE CASCADE
    );

CREATE TABLE Reserved_tickets (
    tid          INT          PRIMARY KEY,
    seat         INT          NOT NULL,
    row          INT          NOT NULL,
    section_name VARCHAR(255) NOT NULL,
    venue_name   VARCHAR(255) NOT NULL,
    FOREIGN KEY (tid) REFERENCES Tickets(tid)
        ON UPDATE CASCADE,
    FOREIGN KEY (seat, row, section_name, venue_name) REFERENCES Seats(seat,
row, section_name, venue_name)
        ON UPDATE CASCADE
);

CREATE TABLE General_tickets (
    tid          INT          PRIMARY KEY,
    section_name VARCHAR(255) NOT NULL,
    venue_name   VARCHAR(255) NOT NULL,
    FOREIGN KEY (tid) REFERENCES Tickets(tid)
        ON UPDATE CASCADE,
    FOREIGN KEY (section_name, venue_name) REFERENCES General_sections(name,
venue_name)
        ON UPDATE CASCADE
);

CREATE TABLE Feature (
    aid          INT          NOT NULL,
    eid          INT          NOT NULL,
    billing_order INT          NOT NULL,
    PRIMARY KEY (aid, eid),
    FOREIGN KEY (aid) REFERENCES Artists(aid)
        ON UPDATE CASCADE,
    FOREIGN KEY (eid) REFERENCES Events(eid)
        ON UPDATE CASCADE
);

CREATE TABLE Review (
    uid          INT          NOT NULL,
    pid          INT          NOT NULL,
    comment      TEXT         NOT NULL,
    event_score  INT          NOT NULL,
    venue_score  INT          NOT NULL,
    PRIMARY KEY (uid, pid),
    FOREIGN KEY (uid) REFERENCES Customers(uid)

```

```

        ON UPDATE CASCADE,
FOREIGN KEY (pid) REFERENCES Performances(pid)
        ON UPDATE CASCADE,
CHECK (event_score >= 1 AND event_score <= 5),
CHECK (venue_score >= 1 AND venue_score <= 5)
);

CREATE TABLE Section_pricetier (
    section_name    VARCHAR(255)    NOT NULL,
    pid             INT              NOT NULL,
    pricetier_name  VARCHAR(255)    NOT NULL,
    PRIMARY KEY (section_name, pid),
    FOREIGN KEY (pricetier_name, pid) REFERENCES Price_tiers(name, pid)
        ON UPDATE CASCADE
);

CREATE TABLE Block (
    pid             INT              NOT NULL,
    seat           INT              NOT NULL,
    row            INT              NOT NULL,
    section_name    VARCHAR(255)    NOT NULL,
    PRIMARY KEY (pid, seat, row, section_name),
    FOREIGN KEY (pid) REFERENCES Performances(pid)
        ON UPDATE CASCADE
);

CREATE TABLE Listings (
    lid             INT              AUTO_INCREMENT PRIMARY KEY,
    tid             INT              NOT NULL,
    seller_id       INT              NOT NULL,
    list_datetime   DATETIME         NOT NULL,
    price           DECIMAL(10, 2)   NOT NULL,
    withdraw_datetime DATETIME       DEFAULT NULL,
    buyer_id        INT              DEFAULT NULL,
    trans_datetime  DATETIME         DEFAULT NULL,
    FOREIGN KEY (tid) REFERENCES Tickets(tid)
        ON UPDATE CASCADE,
    FOREIGN KEY (seller_id) REFERENCES Customers(uid)
        ON UPDATE CASCADE,
    FOREIGN KEY (buyer_id) REFERENCES Customers(uid)
        ON UPDATE CASCADE,
    CHECK (
        (buyer_id IS NOT NULL AND trans_datetime IS NOT NULL AND
withdraw_datetime IS NULL)
        OR (buyer_id IS NULL AND trans_datetime IS NULL)
    )
);

```